SELinux

Use Cases, Real-World Scenarios & Command Reference

A practical guide for RHEL / homelab administrators: what SELinux does, when it matters, three worked scenarios, and an A-to-Z command & term glossary.

Prepared for homelab & RHEL infrastructure work (Ansible, Foreman/Katello, BIND, ESXi)

1. What Is SELinux?

SELinux (Security-Enhanced Linux) is a Mandatory Access Control (MAC) system built into the Linux kernel. Where standard Linux permissions (owner/group/other, DAC) let the resource owner decide who can access it, SELinux adds a second, kernel-enforced layer of policy that even root cannot bypass without explicit policy allowing it. Every process and every file, port, and device carries a security context, and the kernel checks that context against a loaded policy before allowing any action.

Why it exists

- Limits the blast radius of a compromised process — a hacked web server process can only touch what its context is allowed to touch, not the whole filesystem.
- Enforces least-privilege at the kernel level instead of relying only on application-level security.
- Required or strongly recommended in regulated environments (government, finance, healthcare) and is the reason RHEL/CentOS/Fedora ship it enabled by default.

The three modes

Mode	Behavior
Enforcing	Policy is loaded AND enforced. Denials are blocked and logged.
Permissive	Policy is loaded but NOT enforced. Denials are only logged (great for testing/debugging).
Disabled	SELinux is completely off. No labels, no logging, no enforcement (not recommended).

Key building blocks

- **Security Context** — the label attached to every process and object: user:role:type:level (e.g. system_u:object_r:httpd_sys_content_t:s0).
- **Type Enforcement (TE)** — the dominant model in practice: access is decided by matching the process's domain (type) against the object's type.
- **Booleans** — on/off switches that let admins toggle predefined policy behavior without writing new policy (e.g. httpd_can_network_connect).
- **Policy Types** — *targeted* (default; only specific 'targeted' daemons are confined) and *mls* (Multi-Level Security, used in high-assurance environments).
- **Contexts for files vs. running processes** — a file's on-disk label persists; a process's label is inherited/transitioned at exec time.

---

2. Common Use Cases

SELinux is not just a compliance checkbox — it actively prevents whole classes of real incidents. Below are the situations where it earns its keep.

Web server hardening — Confines Apache/Nginx (httpd_t domain) so that even if the web app is exploited (e.g. via a vulnerable PHP upload), the process cannot read /etc/shadow, write to arbitrary system directories, or open unexpected network ports.

Multi-tenant / shared hosting — Prevents one tenant's compromised site from reading another tenant's files, even when both run under the same Linux user via containers or chroots.

Database protection — MySQL/PostgreSQL run in their own domains (mysqld_t, postgresql_t) so a SQL-injection-based shell can't pivot into unrelated services or system files.

Container and virtualization isolation — svirt / container-selinux label each container's files uniquely, so a container breakout doesn't let one container tamper with another's storage on the same host.

Network service confinement — Custom or non-standard ports for SSH, HTTP, Samba, etc. must be explicitly labelled or SELinux blocks the bind/connect — stopping malware from silently listening on unauthorized ports.

File-sharing services (Samba/NFS) — Ensures shared directories are explicitly labelled for sharing (samba_share_t, public_content_t); prevents accidental exposure of unrelated system files through a share.

Regulatory compliance — PCI-DSS, HIPAA, and government (DISA STIG) baselines often mandate SELinux in enforcing mode as a required MAC control.

Privilege-escalation containment — Even a root-owned process confined to a domain cannot perform actions outside its policy — reduces impact of local privilege escalation bugs.

Homelab / Infrastructure-as-Code environments — In Ansible/Foreman/Katello-managed RHEL fleets, SELinux enforces consistent security posture across every provisioned VM without relying on manual hardening per host.

---

3. Three Worked Example Scenarios

Scenario 1 — Apache serving content from a non-default directory

Situation: You move your website's document root from /var/www/html to /srv/www/mysite on a RHEL 9 VM. Apache starts fine, but every request returns '403 Forbidden' even though standard Unix permissions (chmod/chown) are correct.

Why it happens: DAC permissions pass, but the new directory inherited the generic default_t type instead of httpd_sys_content_t, so the httpd_t domain is denied read access by SELinux policy — independent of file permissions.

Diagnosis and fix:

```
# Confirm it's an SELinux denial, not a permissions issue
sudo ausearch -m avc -ts recent
sudo journalctl -t setroubleshoot --since "10 min ago"

# Check current context on the new directory
ls -ldZ /srv/www/mysite

# Set the correct persistent context (survives relabel/reboot)
sudo semanage fcontext -a -t httpd_sys_content_t "/srv/www/mysite(/.*)?"
sudo restorecon -Rv /srv/www/mysite

# Verify
ls -ldZ /srv/www/mysite
sudo systemctl restart httpd
```

Key lesson: chcon changes are temporary and lost on relabel; semanage fcontext + restorecon is the durable, homelab/Ansible-friendly fix (idempotent — safe to run repeatedly in a playbook).

Scenario 2 — SSH or a custom service on a non-standard port

Situation: On your homelab satellite server you configure sshd (or another service, e.g. a custom Foreman/Katello or Ansible-managed API) to listen on port 2222 instead of 22. The service fails to bind with a permission-denied style error even though the firewall (firewalld) is open.

Why it happens: SELinux maintains its own port-type map (separate from firewalld). Only ports explicitly labelled for a service's type (e.g. ssh_port_t) may be bound by that service's domain.

Diagnosis and fix:

```
# See which ports are already associated with a type
sudo semanage port -l | grep ssh

# Add the new port to the correct SELinux type
sudo semanage port -a -t ssh_port_t -p tcp 2222

# If the port were already claimed by another type, modify instead of add:
sudo semanage port -m -t ssh_port_t -p tcp 2222

# Confirm
sudo semanage port -l | grep 2222
sudo systemctl restart sshd
```

Key lesson: Firewall rules (firewalld/iptables) and SELinux port labels are two independent gates — both must allow traffic. This is a very common trip-up when standing up non-standard services in an Ansible-managed homelab.

Scenario 3 — Samba file share silently refusing external access

Situation: You create a Samba share pointing at /data/shared for your homelab so other VMs/clients can pull files. smb.conf looks correct, DAC permissions are 755/rwx as expected, but clients get 'Permission denied' connecting to the share, and a Samba boolean toggle you read about doesn't fully fix it.

Why it happens: Two SELinux controls apply to Samba simultaneously: (1) the directory's file context must allow Samba's domain to read/write it, and (2) an SELinux boolean must permit Samba to share content outside its normal home-directory expectations.

Diagnosis and fix:

```
# Check current label
ls -ldZ /data/shared

# Label the directory for Samba sharing
sudo semanage fcontext -a -t samba_share_t "/data/shared(/.*)?"
sudo restorecon -Rv /data/shared

# List and enable the relevant boolean(s)
getsebool -a | grep samba
sudo setsebool -P samba_export_all_rw on

# Restart the service and re-test from a client
sudo systemctl restart smb
```

Key lesson: File contexts (what the directory is labelled) and booleans (what the confined service's domain is permitted to do in general) are separate layers — a fix often needs both, not just one.

---

4. Command Reference

Grouped by purpose. All commands assume RHEL/CentOS/Fedora with policycoreutils, policycoreutils-python-utils, and setools packages installed.

Status & mode control

Command	Purpose
`getenforce`	Show current mode (Enforcing/Permissive/Disabled).
`setenforce 0 / 1`	Switch live between Permissive (0) and Enforcing (1); not persistent across reboot.
`sestatus`	Detailed status: mode, policy type, mount point, config file mode.
`sestatus -v`	Verbose status including context of key files/processes.
`/etc/selinux/config`	Config file — set SELINUX= and SELINUXTYPE= for persistent boot-time mode.

Viewing contexts

Command	Purpose
`ls -Z`	Show SELinux context of files/directories.
`ps -eZ / ps auxZ`	Show SELinux context of running processes.
`id -Z`	Show your own current SELinux context.
`netstat -Z / ss -Z`	Show contexts associated with network sockets (where supported).

Changing contexts

Command	Purpose
`chcon -t type_t file`	Temporarily change a file's type (lost on restorecon/relabel).
`semanage fcontext -a -t type_t 'path'`	Persistently define the expected context for a path (regex supported).
`semanage fcontext -l`	List all custom file context definitions.
`restorecon -Rv path`	Reapply the defined policy context recursively; the standard 'fix it' command.
`fixfiles onboot`	Schedule a full filesystem relabel on next boot.
`touch /.autorelabel`	Force a full relabel on next reboot (heavy-handed but reliable).

Booleans

Command	Purpose
`getsebool -a`	List all booleans and their current on/off state.
`getsebool boolean_name`	Show state of one specific boolean.
`setsebool boolean_name on`	Toggle a boolean immediately (non-persistent).
`setsebool -P boolean_name on`	Toggle a boolean and persist across reboot.

`semanage boolean -l`	List booleans with description text.

Ports

Command	Purpose
`semanage port -l`	List all port-to-type mappings.
`semanage port -a -t type_t -p tcp PORT`	Add a new port to an SELinux type.
`semanage port -m -t type_t -p tcp PORT`	Modify an already-defined port's type.
`semanage port -d -t type_t -p tcp PORT`	Delete a custom port mapping.

Users, logins & modules

Command	Purpose
`semanage login -l`	List Linux-user-to-SELinux-user mappings.
`semanage user -l`	List SELinux users and their allowed roles.
`semanage module -l`	List installed policy modules.
`semodule -i module.pp`	Install a compiled policy module.
`semodule -r module_name`	Remove an installed policy module.
`semodule -l`	List loaded modules (legacy syntax, still common).

Troubleshooting & logs

Command	Purpose
`ausearch -m avc -ts recent`	Search audit log for recent AVC (Access Vector Cache) denials.
`ausearch -m avc -ts today`	Search denials from today.
`sealert -a /var/log/audit/audit.log`	Human-readable analysis of denials with suggested fixes (setroubleshoot).
`audit2why < denial.log`	Explain why a specific denial occurred.
`audit2allow -a`	Suggest a policy module that would allow observed denials (review before use!).
`audit2allow -a -M mymodule`	Generate a loadable custom policy module from denials.
`journalctl -t setroubleshoot`	View setroubleshoot's plain-language denial summaries.

5. A-to-Z Glossary of Acronyms & Terms

Group	Term	Meaning
A	`AVC`	Access Vector Cache — the kernel's cache of access decisions; AVC denials are the core log entries
A	`audit2allow`	Tool that converts AVC denial log entries into a suggested policy module.
B	`Boolean`	A named on/off switch in policy allowing runtime tuning of allowed behavior without writing new rules.
C	`Context (Security Context)`	The label user:role:type:level assigned to every subject (process) and object (file, port, etc.).
C	`chcon`	Command to temporarily change a file's context (non-persistent).
D	`DAC`	Discretionary Access Control — traditional Unix owner/group/other permissions; SELinux's MAC sits o
D	`Domain`	The 'type' assigned to a running process; determines what that process is allowed to do.
E	`Enforcing`	SELinux mode where policy is actively enforced and violations are blocked.
F	`fcontext`	The persistent file-context database managed via semanage fcontext.
F	`firewalld`	Linux firewall management daemon — independent from, but often confused with, SELinux port labels
G	`getenforce / getsebool`	Utilities to query current mode and boolean states respectively.
H	`httpd_t`	The default SELinux domain/type for the Apache web server process.
I	`Identity (SELinux user)`	The SELinux user component of a context (e.g. system_u, unconfined_u) — distinct from the Linux us
L	`Label`	Colloquial term for a security context attached to a file/process/port.
L	`Level (MLS range)`	The sensitivity level portion of a context, used mainly in MLS policy (e.g. s0).
M	`MAC`	Mandatory Access Control — access rules set centrally by policy, not by resource owners; the model S
M	`MLS`	Multi-Level Security — a stricter SELinux policy type used in high-assurance/government environment
M	`MCS`	Multi-Category Security — category-based isolation (used heavily by containers) layered on top of typ
P	`Permissive`	Mode where denials are logged but not enforced; used for testing new policy.
P	`Policy`	The full rule set loaded into the kernel that governs all SELinux decisions.
P	`policycoreutils`	The RPM package bundling core SELinux userspace tools (semanage, restorecon, setsebool, etc.).
R	`RBAC (Role)`	Role-Based Access Control component of a context — controls which types a given SELinux user ma
R	`restorecon`	Command that resets a file/directory's context to what policy defines as correct.
S	`sealert`	Front-end for setroubleshoot that explains AVC denials in plain language with fix suggestions.
S	`semanage`	The primary administrative tool for managing persistent SELinux policy: file contexts, ports, booleans,
S	`semodule`	Tool for installing, removing, and listing SELinux policy modules.
S	`sestatus`	Displays overall SELinux status and configuration.
S	`setenforce`	Switches between Enforcing and Permissive mode at runtime (non-persistent).

S	`setsebool`	Toggles a boolean's value, optionally persisting it with -P.
S	`setroubleshoot`	Background service that translates raw AVC denials into readable alerts.
S	`svirt`	SELinux extension that isolates virtual machines and containers from each other and the host.
T	`TE (Type Enforcement)`	The dominant SELinux access-control model: rules are written in terms of process types (domains) an
T	`targeted`	The default SELinux policy type on RHEL/Fedora — confines only specific 'targeted' daemons, leaving
T	`Type`	The core attribute in Type Enforcement identifying what 'kind' of process or object something is (e.g. h
U	`unconfined_t`	A domain with very few SELinux restrictions, typically applied to regular user login sessions in targeted

---

Quick recall order: getenforce/setenforce (mode) → ls -Z/ps -Z (inspect) → semanage fcontext + restorecon (fix file labels) → semanage port (fix ports) → getsebool/setsebool (fix booleans) → ausearch/sealert/audit2allow (diagnose & generate policy).